



Table of contents

1. Overview
2. Contact
3. Disclaimer
4. Installation notes
5. Running the software
6. Basic operation



1. Overview

P2ROUD = Preclinical PROspective Undersampling in multiple Dimensions

P2ROUD is a software utility and graphical user interface to reconstruct MRI data acquired with an MR Solutions preclinical MRI system, in particular data with k-space undersampling in one or more dimensions. Besides Cartesian 2D and 3D data it reconstructs radial, 3D UTE and ZTE scans, and EPI scans for diffusion and fMRI, which it can export as a BIDS dataset.

2. Contact

P2ROUD and this manual are written by:

Gustav Strijkers
Amsterdam UMC
Biomedical Engineering and Physics
Amsterdam, the Netherlands
g.j.striekers@amsterdamumc.nl

and

Bram Coolen
Amsterdam UMC
Biomedical Engineering and Physics
Amsterdam, the Netherlands
b.f.coolen@amsterdamumc.nl

3. Disclaimer

The software has been extensively tested using mouse and rat data acquired with an MR Solutions preclinical 7.0T/24cm system.

However, this does not warrant the functions contained in the program will meet your requirements or that the operation of the program will be uninterrupted or error-free.

In case of questions or issues, please contact Gustav Strijkers or Bram Coolen.



4. Installation notes

Software download

The Matlab app and a Windows standalone version (using the free Matlab runtime engine) can be downloaded from GitHub:

<https://github.com/Moby1971?tab=repositories>

Installation of the Windows standalone version

`P2ROUD_app_installer.exe`

Will install the Matlab runtime engine and the P2ROUD program.

Bart toolbox download

The advanced reconstruction options in P2ROUD require the Bart reconstruction toolbox, which can be downloaded from:

<https://mrirecon.codeberg.page/>

P2ROUD is tested with Bart 1.0.00 (its source, `functions/bart/bart-1.0.00-master.zip`, is not in the GitHub download). It looks for Bart in `TOOLBOX_PATH`, then in `/usr/local/bin` and `/usr/bin`, and on Windows in the default WSL distribution. The message box says at start whether Bart is found and built for the GPU, which it then uses.



Bart toolbox installation in OSX with MacPorts

(1) Install Xcode from the Mac App Store

```
$ xcode-select --install
```

(2) Install MacPorts (<http://www.macports.org/>)

It is recommended to install a newer version of gcc from MacPorts

(3) Install packages

```
$ sudo port install fftw-3-single
$ sudo port install gcc14
$ sudo port install libpng
$ sudo port install openblas
$ sudo port install flock
$ sudo port install gmake
$ sudo port install coreutils
$ sudo port install libblocksruntime
```

(4) Make

```
$ export PATH=/opt/local/libexec/gnubin:$PATH
$ CC=gcc-mp-14 gmake
$ sudo gmake install
```

Bart toolbox installation in OSX with HomeBrew

(1) Install Xcode from the Mac App Store

```
$ xcode-select --install
```

(2) Install HomeBrew

```
$ /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

(3) Install packages

```
$ brew install libpng
$ brew install fftw
$ brew install openblas
$ brew install gmake
$ brew install llvm libomp
```

(4) Make

```
$ CC=gcc MACPORTS=0 gmake
$ sudo gmake install
```



Bart toolbox installation in Windows 11

(1) Install the Windows subsystem for Linux

Start a windows powershell as administrator and run the following command:

```
Enable-WindowsOptionalFeature -Online -FeatureName Microsoft-Windows-Subsystem-Linux
```

A system restart will be needed.

For more information, see:

<https://pureinfotech.com/install-wsl-windows-11/>

(2) Download Ubuntu Linux

Open PowerShell

To see a list of available Linux distributions for download, enter:

```
wsl -l -o
```

Install a Linux distribution:

```
wsl --install -d <distribution name>
```

You will be asked to create a user account the first time you start a Linux command prompt.

(3) Upgrade to WSL2 if needed

(see e.g. <https://pureinfotech.com/install-wsl-windows-11/> on how to upgrade).

(4) Upgrade the Linux distribution

```
$ sudo apt-get update
$ sudo apt-get dist-upgrade
```

(5) Install Bart prerequisites (gcc 12 or newer)

```
$ sudo apt-get install make gcc libfftw3-dev liblapacke-dev libpng-dev
libopenblas-dev gfortran
```

(6) Download Bart

```
$ wget https://github.com/mrirecon/bart/archive/v1.0.00.tar.gz
```

(7) Build Bart, or with GPU support as in step (8)

```
$ tar xvfz v1.0.00.tar.gz
$ cd bart-1.0.00
$ make
$ sudo make install
```



(8) Build Bart with GPU support (highly recommended)

- Install the NVIDIA driver in Windows, as described at <https://docs.nvidia.com/cuda/wsl-user-guide/index.html>. It is what gives WSL the GPU; no Linux display driver goes inside WSL.
- Install the CUDA toolkit inside WSL, from <https://developer.nvidia.com/cuda-downloads> (Linux, x86_64, WSL-Ubuntu). For CUDA 12.8:

```
$ wget https://developer.download.nvidia.com/compute/cuda/12.8.0/local_installers/cuda_12.8.0_570.86.10_linux.run
$ sudo sh cuda_12.8.0_570.86.10_linux.run
```

- Set the environment variables, in the shell you build in: CUDA_BASE for the CUDA version installed, GPUARCH_FLAGS for the card. Bart has to be built for the card's generation (look it up at <https://developer.nvidia.com/cuda-gpus>): a build for another one compiles without complaint and fails at the first calculation.

generation	cards	GPUARCH_FLAGS
Pascal	GTX 1060, 1080, Titan X	-arch=sm_61
Turing	RTX 2060 to 2080	-arch=sm_75
Ampere	RTX 3060 to 3090	-arch=sm_86
	A100	-arch=sm_80
Ada Lovelace	RTX 4060 to 4090	-arch=sm_89
Blackwell	RTX 5070 to 5090	-arch=sm_120

```
$ export CUDA=1
$ export CUDA_BASE=/usr/local/cuda-12.8
$ export CUDA_LIB=lib64
$ export GPUARCH_FLAGS="-arch=sm_61"
$ export PATH=$CUDA_BASE/bin:$PATH
$ export LD_LIBRARY_PATH=$CUDA_BASE/$CUDA_LIB:$LD_LIBRARY_PATH
```

- Make. If the linking fails with an error about libstdc++ or libgcc, point the linker at the installed gcc with the last line (its number from `ls /usr/lib/gcc/x86_64-linux-gnu/`) and make again.

```
$ make -j$(nproc) CUDA=1 CUDA_BASE=$CUDA_BASE CUDA_LIB=$CUDA_LIB
    GPUARCH_FLAGS="$GPUARCH_FLAGS"
$ sudo make install
$ export LDFLAGS="-L/usr/lib/gcc/x86_64-linux-gnu/13 $LDFLAGS"
```

- Test

```
$ make utest_gpu
```



5. Running the software

Running in Matlab 2025a

The P2ROUD software can be started from its root directory from the command line.

```
>> P2ROUD
```

P2ROUD starts in about 12 seconds. MATLAB needs about 5 GB of memory for a ZTE scan of 250 dynamics, 7 GB for a DTI scan of 130 volumes and 13 GB for an fMRI scan of 1501 dynamics.

Additional licenses may be required.

```
MATLAB
Image Processing Toolbox
Parallel Computing Toolbox
Signal Processing Toolbox
Statistics and Machine Learning Toolbox
Wavelet Toolbox
```

Running the Windows standalone

The Windows standalone version can be run from the start menu or the desktop icon.

At start, P2ROUD checks online for a newer version and looks for Bart; the message box reports both. The first Load MRD dialog opens in the scanner's MRD folder, or in the folder written in mrdpath.txt, a text file in the folder P2ROUD is started from.

Settings and log files

P2ROUD keeps its settings in a file, p2roudSettings.json, in a folder of its own for every version of the app:

```
macOS      ~/Library/Application Support/P2ROUD/<version>
```

```
Windows    %APPDATA%\P2ROUD\<version>
```

The file is written with the standard values the first time a version starts, and can then be edited by hand; changes take effect at the next start. It holds, among others, the default compressed sensing weights and iterations for each type of data, whether the EPI corrections and the ZTE registration are ticked at start, and the field strength and animal weight written into Dicom headers made without the scanner's own (7 T and 0.03 kg, a mouse). A value that cannot be used is replaced by its standard value, and the message box says so.

Every session writes a log file, p2roud-log-<date and time>.txt, to the same folder. The log keeps every message, also those the message box does not show at the chosen message level. The folder also holds ktraj.txt, the 3D UTE readout profile (see Step 1).



6. Basic operation

The P2ROUD program operates from a single window with 5 panels.



Panel 1: Dashboard

This panel contains the task buttons and parameters that control the reconstruction process. The dashboard tasks need to be completed from top to bottom. A green light next to the task indicates that the task has been completed. When the light is yellow the task is busy. Red indicates not completed yet, or failed.

Panel 2: K-space / Coils

This panel shows the raw k-space data, and the coils and their sensitivity maps.

Panel 3: Scan parameters / Geometry / Reconstruction / Radial

This panel displays the relevant acquisition, geometry, reconstruction and radial reconstruction parameters.

Panel 4: Images / MIP / Phase

After reconstruction is completed this panel shows the reconstructed images.

Panel 5: Messages / Settings

Program status and messages. In the Settings tab the sounds can be switched on or off, the classic or black/white colors chosen, and the message level (1 to 3) set: how much the message box shows. The status icon shows OK, WARNING, ERROR or BUSY.



Step 1: Loading data

Press  Load MRD to load the MRD raw data file.

P2ROUD can reconstruct spin-echo, fast spin-echo, gradient-echo, MPRAGE and CINE data, multi-slab 3D, 2D radial, 3D UTE and ZTE data, and 2D EPI (diffusion and fMRI).

However the program is particularly designed to reconstruct undersampled data with user-defined undersampling pattern (e.g. SENSE, random undersampling, pseudo-spiral or radial).

For data acquired with multiple receivers (phased-array RF coil) select 1 of the MRD files.

Relevant acquisition parameters will be shown in panel 3.

k-space trajectory

There are three options to include a user-defined k-space trajectory in the FLASH (version 635e) sequence.

(1) 2D-data

pe1-order = 3 ("LUT")

The k-space points for the 1st phase-encoding direction (views) are provided in the views-table in the sequence.

(2) 3D-data

pe1-order = 4 ("exLUT")

The k-space points for the 1st and 2nd phase-encoding directions (views and views2) are provided in a text file (*.txt) as a list of points without comma separation.

For example:

```
-48      (views      k-point 1)
0        (views2     k-point 1)
-47      (views      k-point 2)
0        (views2     k-point 2)
-46      .
1        .
-45      .
.
.
etc...
```

(3) Undersampled 2D- and 3D-data

pe1-order = 5 ("nrLUT")

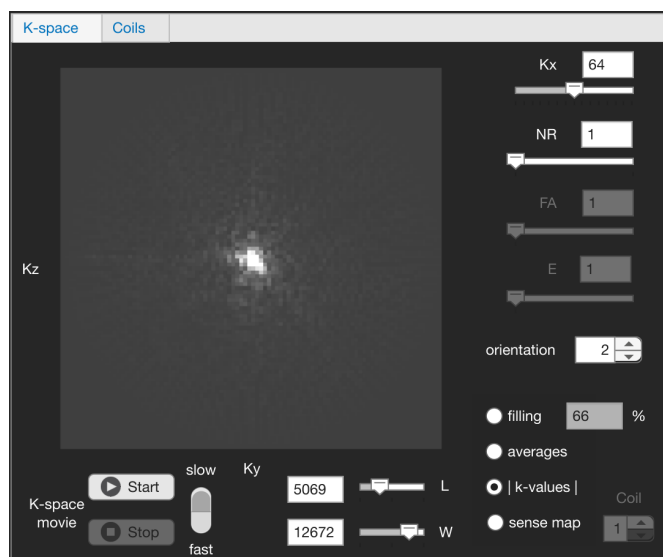
Trajectories are read from the first .txt file in the folder of the MRD file (3D UTE: lut*.txt).

The readout profile, kttraj.txt in the settings folder, has one column for all axes or three for x, y and z. MATLAB and Python code to make trajectories is in the trajectoryTools folder.



Step 2: Inspecting k-space

The raw k-space data is displayed in panel 2.

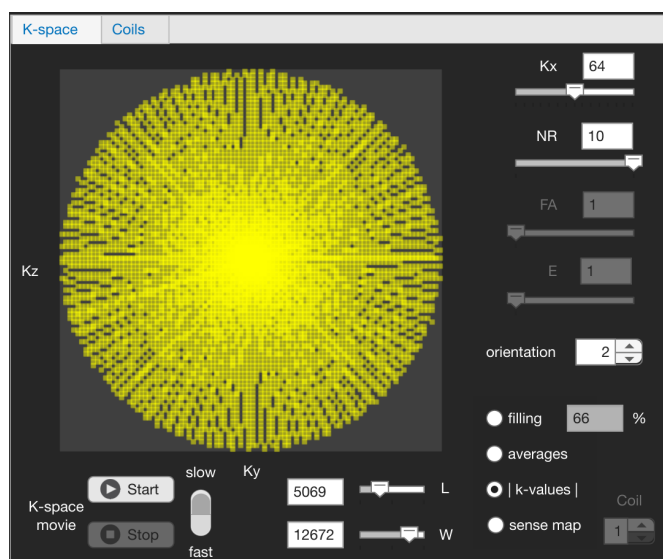


The window displays the acquired k-space points (filling), the averages per k-point (averages), the absolute value of k-space ($|k\text{-values}|$) or a coil sensitivity map (sense map). The percentage of k-space points acquired is indicated (66% for this dataset).

The sliders step through the 3rd k-space dimension, slices, repetitions (**NR**), flip angles (**FA**) and echoes (**E**), if applicable. The orientation spinner chooses the k-space plane shown.

Step 3: Displaying the k-space trajectory

Press  to start an animation of the k-space trajectory. An example for a pseudo-radial k-space trajectory is shown below



Press  to stop the animation. Press



to toggle between fast and slow animation speed.



Step 4: Fourier transform reconstruction

For a first quick inspection of the reconstructed images, a Fourier transform reconstruction of the data can be done by pressing  .



Dimensions

X 128

Y 128

Slices 15

NR 1

Reconstruction

 Reco 

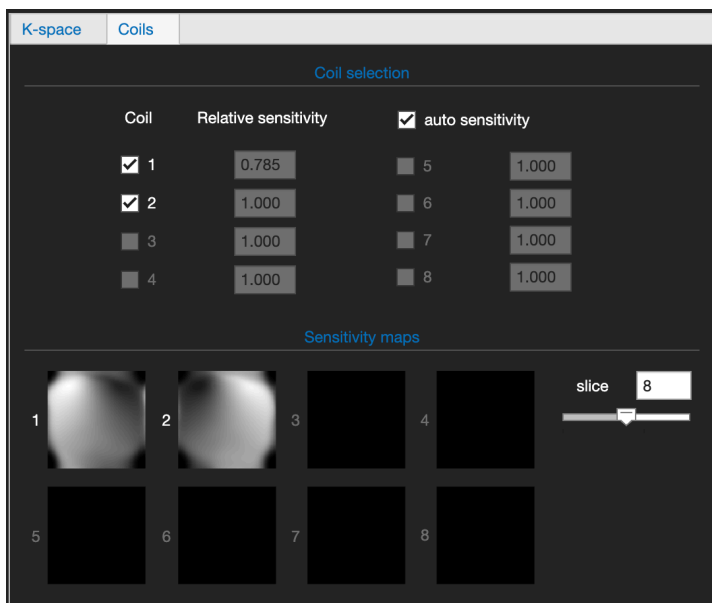
 0 20 40 60 80 100 

CS reco ☐

X, Y and Z in the Dimensions panel set the size of the images: for Cartesian data k-space is zero-filled or cropped, for radial, UTE and ZTE data the images are resized. For 2D data the third field, Slices, keeps the number of slices acquired.

Step 5: Coils panel

For acquisitions with a phased-array coil the Coils tab gives access to the channels: coils can be checked or unchecked to use or omit them in the reconstruction. Relative sensitivities can be set by hand, or determined with auto sensitivity, which for a CS reconstruction with Bart also computes the ESPIRiT coil maps shown under Sensitivity maps. Auto sensitivity is ticked again at every load, change of NR and reconstruction.



K-space **Coils**

Coil selection

Coil	Relative sensitivity	☒ auto sensitivity	
☒ 1	0.785	☐ 5	1.000
☒ 2	1.000	☐ 6	1.000
☐ 3	1.000	☐ 7	1.000
☐ 4	1.000	☐ 8	1.000

Sensitivity maps

1 2 3 4

5 6 7 8

slice 8



Step 6: Reconstruction

Checking the **CS reco** box gives access to the Compressed Sensing (CS) reconstruction parameters.

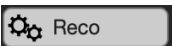
The image shows two tabs: 'Dimensions' and 'Reconstruction'. In the 'Dimensions' tab, there are input fields for X (128), Y (128), Slices (15), and NR (1). In the 'Reconstruction' tab, there is a 'Reco' button with a gear icon, a green status light, a progress bar from 0 to 100, an 'abort' button, and a 'CS reco' checkbox which is checked.

The image shows the 'Reconstruction' tab with three sub-sections: 'Compressed sensing', 'EPI', and 'Denoising / filtering'. Under 'Compressed sensing', there are input fields for WVxyz (0.0010), TVxyz (0.0000), LLRxyz (0.0000), TVtime (0.0000), and CS iterations (500). Under 'EPI', there are checkboxes for distortion corr. (checked), POCS (checked), and registration (checked). Under 'Denoising / filtering', there are checkboxes for PCA (unchecked), BMD (unchecked), deRing (unchecked), and Tukey (unchecked). At the bottom, there are 'Sum acquisitions' fields for Σ NR (1) and Σ NE (1 1 1 1 1 1 1 1 1 1), each with a 'SoS' dropdown menu.

The CS regularization parameters are:

- (1) **WVxyz**
Wavelet regularization parameter in the spatial (x, y, and z) dimensions.
- (2) **TVxyz**
Total variation constraint in the spatial dimensions.
- (3) **LLRxyz**
Locally Low Rank constraint in the spatial dimensions.
- (4) **TVtime**
Total variation constraint in the dynamics (repetitions / **NR**) dimension.

The spatial dimensions of the reconstructed data can be increased or decreased. The number of repetitions (**NR**) can also be changed to undersample or average data in the dynamics dimension.

Pressing  will start the CS reconstruction with the BART toolbox.

Without the BART toolbox, Cartesian data are reconstructed with a slower CS algorithm implemented in Matlab (WVxyz and TVtime only), and radial data without CS.

Denoising, the Tukey filter, the sums over dynamics and echoes, and the EPI corrections are set in the same tab; they are described on the next page.

For multi-slab 3D (MOTSA) scans, the slabs are put where they were acquired and blended over slab overlap (pixels), in the Geometry tab, on either side of each junction.



Further options in the Reconstruction tab

CS iterations sets the number of iterations of the radial, UTE and ZTE reconstructions with Bart. A running reconstruction can be stopped with abort.

Denoising is applied to the reconstructed images; the three methods can be combined. PCA removes noise by principal component analysis with the Marchenko–Pastur criterion (Cordero-Grande et al., NeurolImage 2019), over the images of each voxel in a series (dynamics, diffusion directions, flip angles, echoes), in a window chosen from the data; for EPI it runs before the EPI corrections. BMD applies block-matching denoising: BM3D per slice for 2D data, BM4D per volume for 3D, UTE and ZTE data. deRing suppresses Gibbs ringing. Tukey tapers the edges of k-space against ringing, at some cost in resolution; loading a Cartesian scan unticks it, a radial or UTE scan ticks it.

With Σ NR or Σ NE ticked, the dynamics or the echoes are combined into one image by SoS, Max or Sum; the field beside the box selects which, with a 1 or 0 for each.

EPI scans

2D EPI scans, single-shot or read-interleaved, spin echo or gradient echo, with or without partial Fourier, are reconstructed as diffusion series (DTI, several b-values) or fMRI time series. Ghost and drift correction are always applied, and masking when the study has a FLASH mask scan. Three corrections in the EPI part of the tab are on by default:

distortion corr. corrects the geometric distortion with a partner scan of the same study, with the same slices and echo train and the opposite phase-encoding direction, which the app finds by itself: a displacement field fitted to the physical distortion model is applied to every volume. Without a partner scan the message box says so and nothing is changed.

POCS completes a partial Fourier scan: sharper images, at some cost in signal-to-noise ratio.

registration aligns the volumes with the mean b=0 volume (diffusion) or the first volume (fMRI), by shifts in the image plane.

The DCM / NIFTI export of an EPI series also writes a BIDS dataset in BIDS/sub-<study>/ses-1: func (fMRI) or dwi with the gradient table (diffusion), and anat, a fast spin echo scan of the study that the app finds and reconstructs itself (T2w for a TE above 30 ms, otherwise T1w); an fMRI series needs this scan, a diffusion series does not.

For a diffusion scan with at least 6 directions and a b=0 volume, MRtrix3 and DSI Studio scripts for tractography (tractography_MRtrix_<tag> and tractography_DSIStudio_<tag>, .sh and .bat) are written into the NiftI folder; the app does not run them. The DTI and fMRI pipelines are described in the manuals folder.



Reconstruction of radial scans

The *Radial* tab gives access to advanced reconstruction parameters for 2D radial, 3D UTE and ZTE scans.

The screenshot shows the 'Radial' tab in the software interface. It is divided into three main sections: Gradient delays, Corrections, and ZTE. The Gradient delays section includes a 'calibration' checkbox (unchecked) with an 'abort' button, a 'ring method' dropdown (set to 'Bart'), and input fields for Gx delay, Gy delay, Gz delay, Sxy delay, Sxz delay, Syz delay, and data offset (set to 8). The Corrections section includes checkboxes for 'center echo' (unchecked), 'phase correction' (unchecked), 'amplitude correction' (unchecked), 'density correction' (checked), and 'large FOV' (unchecked). The ZTE section includes input fields for 'dead time (μs)' (set to 11.5), 'shift (samples)' (set to 4), and checkboxes for 'full echo' (unchecked) and 'registration' (unchecked).

Tick the calibration checkbox to estimate the gradient delays from the data at the next reconstruction; the delay fields are then filled in and the checkbox is unticked. With Bart, 2D radial scans load with calibration and ring method ticked. With ring method ticked the delays are estimated with Bart's *estdelay*, using the method of Block and Uecker; RING, after which the checkbox is named, is not used, as it is unreliable on 0–180° trajectories. Otherwise, and for 3D UTE, an iterative low-rank method is used, which is much slower and does not need Bart; abort stops it. For 3D UTE the calibration also estimates data offset, the number of samples dropped from the start of each readout. The delays, in samples, are Gx, Gy and Sxy for 2D radial data, and Gx, Gy, Gz, Sxy, Sxz and Syz for UTE and ZTE; they are reset when a data set is loaded.

Center echo centers the echoes before reconstruction and *phase correction* corrects their phase; a 3D UTE scan loads with both unticked and data offset 8.

Large FOV reconstructs a 3D UTE scan over 1.5 times the field of view and crops it, so that less signal leaks in from beyond the field of view; it takes about a quarter longer.

Density correction performs a 2D or 3D density correction in k-space to account for non-uniform sampling. This setting only applies to reconstructions with *Bart*.

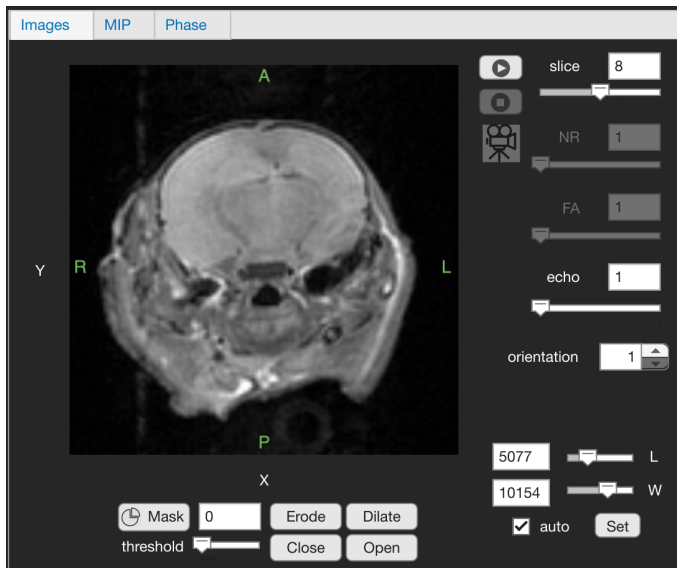
Amplitude correction scales every spoke to the mean peak amplitude. The Tukey filter, which tapers the readout, is set in the Reconstruction tab.

ZTE scans have no calibration and none of these corrections. Their dead time (μs) is the delay between the excitation and the first sample, 11.5 μs by default, and shift (samples) the first sample kept, 4 by default; changing shift rebuilds k-space. With full echo both halves of the echo are used, and registration aligns the dynamics of a series in 3D.



Step 7: Viewing the reconstructed images

After reconstruction the MR images are displayed in panel 4.

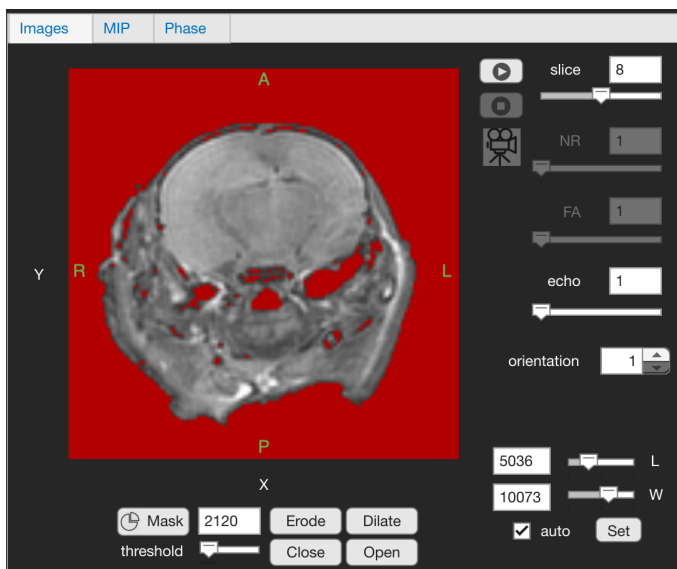


The image orientation can be changed with the orientation spinner; the green letters around the image give the anatomical directions.

Image intensity can be adjusted with the **Level** and **Window** sliders. Checking auto adjusts the intensity automatically.

Press  to start a movie of images as function of the dimension / parameter shown next to the camera icon .

Images can be segmented (on the basis of a threshold) by pressing .



Use the threshold slider or the morphological operations (Erode, Dilate, Open, Close) to fine-tune the segmentation. The mask is used for display and phase unwrapping only.



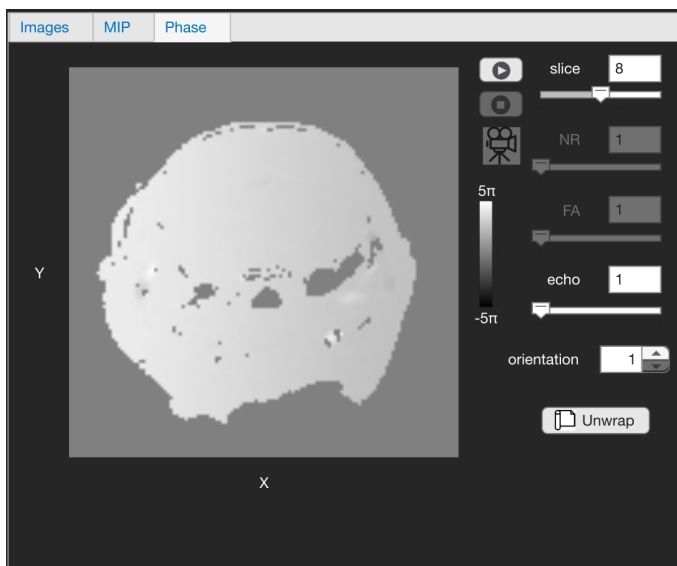
Maximum intensity projection (MIP)

In the *MIP* tab a maximum intensity projection is shown. The MIP can be rotated and animated using the available sliders and buttons; Reset returns to the start and Resize halves the volume for speed.



Phase images

The phase tab shows the phase images.



Phase can be unwrapped by pressing the  Unwrap button.



Step 8: Image export

There are several ways to export the images.



(1) DCM / NIFTI

Exports the images as Dicom files for further processing in 3rd party software. When the scanner's own Dicom images are found, their headers are used; otherwise the headers are made from the MRD data, with the orientation taken from the scan angles and the position set to the isocenter.

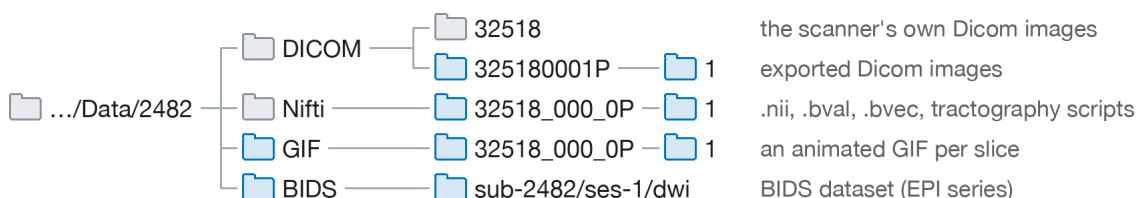
The images are also written as Nifti files with their geometry, and for diffusion scans with .bval and .bvec files; recoparameters_<tag>.txt records the scan and the settings used. For EPI series a BIDS dataset and scripts for tractography are written as well (see "EPI scans"). The files go into .../Data/<study>/DICOM/<series>P/<n> and Nifti/<tag>P/<n>, a new numbered folder <n> each time, when the scanner's Dicom export of the scan is found in .../Data/<study>/DICOM/<scan>; otherwise into a folder that is asked for.

(2) MRD

The data will be stored as a new MRD file, p2roud_<n>_<tag>.MRD, in the folder of the input MRD file (or in a folder asked for, when that cannot be written), with recoparameters_<tag>.txt and, if the program finds an input rpr file, a new rpr file. The scan's rtable.rtv is renamed origtable.rtv.orig. Additionally, when the program runs on the scanner computer, data will be reconstructed by the scanner software and new SUR files, and for nrLUT scans Dicom images made from them, will be placed in the data directory for viewing and planning of next scans in the MR Solutions preclinical software. nrLUT scans are exported to MRD by themselves after every reconstruction.

(3) GIF

The data will be exported as (animated) GIF files. When the MRD file lies in .../MRD/<study>/<scan>, they go into .../Data/<study>/GIF/<tag>P/<n>, a new numbered folder each time; otherwise into a folder that is asked for.



Step 9: Exit

Press  Exit to shut down the program. Closing the window does the same, also while the program is busy.



Credits

- BART toolbox and compressed sensing algorithms

<https://mrirecon.codeberg.page/>

Michael Lustig, David Donoho, John M. Pauly, Sparse MRI: The application of compressed sensing for rapid MR imaging, Magn Reson Med 58, 1182-1195 (2007).

- Automatic gradient delay estimation algorithm

Block, K. T. & Uecker, M., Simple method for adaptive gradient-delay compensation in radial MRI. Proc. Intl. Soc. Mag. Reson. Med. 19, 2816 (2011)

- Gibbs ringing suppression

Kellner, E, Dhital B., Kiselev VG and Reiser, M., Gibbs-ringing artifact removal based on local subvoxel-shifts. Magn Reson Med, 76(5), 1574-1581.

- NUFFT

<https://web.eecs.umich.edu/~fessler/>

- Phase unwrapping

M. A. Herráez, D. R. Burton, M. J. Lalor, and M. A. Gdeisat, "Fast two-dimensional phase-unwrapping algorithm based on sorting by reliability following a noncontinuous path", Applied Optics, Vol. 41, Issue 35, pp. 7437-7444 (2002),

H. Abdul-Rahman, M. Gdeisat, D. Burton, M. Lalor, "Fast three-dimensional phase-unwrapping algorithm based on sorting by reliability following a non-continuous path", Proc. SPIE 5856, Optical Measurement Systems for Industrial Inspection IV, 32 (2005).

- Denoising

- Y. Mäkinen, L. Azzari, A. Foi, 2020, "Collaborative Filtering of Correlated Noise: Exact Transform-Domain Variance for Improved Shrinkage and Patch Matching", in IEEE Transactions on Image Processing, vol. 29, pp. 8339-8354.

- M. Maggioni, V. Katkovnik, K. Egiazarian, A. Foi, 2013, "Nonlocal Transform-Domain Filter for Volumetric Data Denoising and Reconstruction", in IEEE Transactions on Image Processing, vol. 22, pp. 119-133.

- Y. Mäkinen, S. Marchesini, A. Foi, 2022, "Ring Artifact and Poisson Noise Attenuation via Volumetric Multiscale Nonlocal Collaborative Filtering of Spatially Correlated Noise", in Journal of Synchrotron Radiation, vol. 29, pp. 829-842.